LAB2 Частина 2. Оброблення даних про файли

Виконавець: **Тижук Дмитро Іванович**, група **К-26**
Викладач: **Ляшко А. В.**

Нам знадобиться бібліотека `pandas`.
Бібліотека є сторонньою, отже потребує встановлення. Краще у віртуальне оточення.
Мала бути встановлена, якщо ні (клітина з імпортуванням виконується з помилкою), то треба встановити. Можна прямо з Jupyter. Тільки не під час пари.

Наступну клітину можна розкоментовувати та виконувати тільки, якщо в якості ядра (kernel) вибране віртуальне оточення!

In [1]:
```
#!pip install pandas
```

Якщо встановлювати не в оточення, то хоча б тільки для користувача (з ключем --user).

In [2]:
```
#!pip3.14 install pandas --user    # Linux
```

In [3]:
```
#!py -3.14 -m pip install pandas --user    # Windows
```

In [4]:
```
from pathlib import Path
import pandas as pd

print(f"pandas {pd.__version__}")
```

```
pandas 3.0.1
```

Постановка задачі

Згадка про минуле (Лабораторну роботу №1)
Протягом навчання програмні проєкти з різних предметів зберігалися в папках з переліку `folders`, який оброблявся в лабораторній роботі №1.
Щоб профільтрувати допоміжні файли середовищ, було введено "білий список" розширень (`WHITE_LIST`) та список розширень файлів з текстом програм(задається) (`CODE_LIST`).
Також було написано функцію `kind`, що за розширенням повертає рядок з символом C,W або O відповідно для файлів з текстом програм (відповідно до переліку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список).

У результаті виконання лабораторної роботи для кожного місяця та кожного розширення, що зустрічаються при перегляді файлів, розміщених у папках зі списку, було знайдено таку інформацію:

```
рік, місяць, розширення, кількість файлів, сумарний обсяг
у байтах, кількість їх рядків.
```

Кількість рядків наводиться тільки для розширень категорії C; білі та порожні рядки не рахуються; для інших розширень ця клітина таблиці залишається незаповненою.

За цією інформацією було побудовано csv-файл з іменем `lab1_1.csv` (кодування тільки utf-8), на початку якого розміщено рядок з заголовками: year, mon, ext, cnt, size, lines.

In [5]:
```
CODE_LIST = {".cpp", ".hpp", ".c", ".h", ".py", ".pyw", ".cs", ".js", ".j
WHITE_LIST = CODE_LIST | {".html", ".json", ".txt", ".ipynb"}
```

In [6]:
```
headers = ["year", "mon", "ext", "cnt", "size", "lines"]
```

За інформацією з csv-файлу з'ясувати

1. Скільки різних розширень зустрічається у папках з заданого списку, скільки файлів з кожним розширенням. (Для цієї задачі розглядаємо всі розширення, а не тільки "білого списку".)
   Відповідну інформацію вивести на екран у табличному вигляді. Рядки таблиці містять розширення, кількість файлів та ознаку (C,W,O відповідно для файлів з текстом програм (розширення з списку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.
2. За датами створення файлів для кожного місяця, починаючи з вересня 2024-го року, знайти таку інформацію: кількість файлів з розширеннями з "білого списку", їх сумарний розмір у кілобайтах, кількість їх рядків (враховувати тільки для файлів категорії C), середній розмір у байтах, середню кількість рядків.
   Відповідну інформацію вивести на екран у табличному вигляді в порядку, оберненому до хронологічного.

Зверніть увагу на підписи стовпчиків, що будуть виводитись.

Всі значення округлити та привести до цілого типу.

Завантаження даних

Задаємо шлях до файлу `lab1_1.csv`.

In [7]:
```
path = Path("./lab1_1.csv")
```

Завантажимо дані за допомогою функції `pd.read_csv` (документація тут).

Зверніть увагу на те, що:

1. файл записано в кодуванні `utf-8`

2. перший рядок файлу містить заголовки year, mon, ext, cnt, size, lines
3. значення у стовпчику `lines` можуть бути відсутні

▶ Підказки.

▶ Ще більше підказок

In [8]:
```
data = pd.read_csv(
    path,
    encoding="utf-8",
    header=0,
    sep=",",
    usecols=[
        "year",
        "mon",
        "ext",
        "cnt",
        "size",
        "lines",
    ],
    dtype={"lines": "Int64"},
    skipinitialspace=True,
)
data.head(3)
```

Out[8]:

	year	mon	ext	cnt	size	lines
0	2025	2	.txt	19	60336	<NA>
1	2025	2	.h	5	2139	91
2	2025	2	.cpp	13	119213	3273

In [9]:
```
data.tail(3)
```

Out[9]:

	year	mon	ext	cnt	size	lines
74	2026	2	.c	1	0	0
75	2026	2	.html	2	20301	<NA>
76	2026	2	.txt	8	97417	<NA>

Подивимось інформацію про завантажені дані. Документація на метод `info`.

In [10]:
```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 77 entries, 0 to 76
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   year    77 non-null     int64
 1   mon     77 non-null     int64
 2   ext     77 non-null     str
 3   cnt     77 non-null     int64
 4   size    77 non-null     int64
 5   lines   47 non-null     Int64
dtypes: Int64(1), int64(4), str(1)
memory usage: 3.8 KB
```

Бачимо, що відсутні значення є не тільки в стовпчику `lines`, що природньо, бо кількість рядків обчислювалась не для всіх розширень, але і в стовпчику `ext` з розширеннями.

Замінимо відсутні розширення порожнім рядком.

Використаємо метод `fillna`.

Слід замінити тільки відсутні розширення, тому значення параметра `value` слід зробити словником, що для кожного стовпчика, який слід оброблювати, задає значення-заповнювач.
Не забудьте, що перетворення слід виконувати "на місці".

▶ Підказка (тільки за повного безвихіддя)

Альтернатива. Можна спробувати перетворити тільки стовпчик `data.ext`.

▶ Підказка (тільки за повного безвихіддя)

Перший спосіб більш економний за пам'яттю. Другий спосіб корисний тоді, коли для різних стовпчиків використовують різні алгоритми заповнення відсутніх значень.

In [11]:
```
data.fillna(value={"ext": ""}, inplace=True)
data.head(3)
```

Out[11]:

	year	mon	ext	cnt	size	lines
0	2025	2	.txt	19	60336	<NA>
1	2025	2	.h	5	2139	91
2	2025	2	.cpp	13	119213	3273

In [12]:
```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 77 entries, 0 to 76
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   year    77 non-null     int64
 1   mon     77 non-null     int64
 2   ext     77 non-null     str
 3   cnt     77 non-null     int64
 4   size    77 non-null     int64
 5   lines   47 non-null     Int64
dtypes: Int64(1), int64(4), str(1)
memory usage: 3.8 KB
```

Тепер незаповнені дані залишились тільки у стовпчику `lines.`

Ні в якому разі там не повинно бути нулів!

Групування та групові операції

In [13]:
```
data.head()
```

Out[13]:

	year	mon	ext	cnt	size	lines
0	2025	2	.txt	19	60336	<NA>
1	2025	2	.h	5	2139	91
2	2025	2	.cpp	13	119213	3273
3	2025	2	.json	33	205855	<NA>
4	2025	2	.c	4	110480	3000

Задача 1 Скільки різних розширень зустрічається в папках з заданого списку, скільки файлів з кожним розширенням?

Відповідну інформацію вивести на екран у табличному вигляді. Рядки таблиці містять розширення, кількість файлів та ознаку (C,W,O відповідно для файлів з текстом програм (розширення з списку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.

Скільки різних розширень зустрічається в папках з заданого списку?

Найпростіше: взяти стовпчик таблиці, що зберігає розширення, та знайти кількість унікальних значень.

▶ Підказки

In [14]:
```
data["ext"].nunique()
```

Out[14]: 11

Скільки файлів з кожним розширенням?

Відповідну інформацію вивести на екран у табличному вигляді. Рядки таблиці містять розширення, кількість файлів та ознаку (C,W,O відповідно для файлів з текстом програм (розширення з списку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.

Спочатку треба підсумувати інформацію по кожному розширенню. Допоможе метод `groupby` документація тут.

Групувати слід за розширенням (параметр `by`) і для кожної групи обчислити суму кількостей файлів (метод `sum`).

Щоб не сумувати роки, місяці тощо, групування слід застосувати до вибірки потрібних стовпчиків. Маємо вибрати стовпчик, за яким групуємо, і стовпчик, який сумуємо.

▶ Підказка

Результат присвоїти змінній `task1`.

In [15]:
```
task1 = data[["ext", "cnt"]].groupby(by="ext").sum()
task1
```

Out[15]:

ext	cnt
.c	45
.cpp	92
.cs	692
.h	146
.hpp	23
.html	13
.ipynb	20
.js	158
.json	421
.py	9499
.txt	369

Відсортувати за спаданням кількості файлів. Допоможе метод `sort_values` (документація тут).

Не забудьте вказати, що сортування відбувається за спаданням та "на місці".

▶ Підказки

In [16]:
```
task1.sort_values(by="cnt", inplace=True, ascending=False)
task1.head(3)
```

Out[16]:

ext	cnt
.py	9499
.cs	692
.json	421

За умовою ще треба додати стовпчик з ознакою категорії розширення файлу W,C,O (див. умову). У лабораторній роботі №1 була написана функція `kind`, що за розширенням повертає рядок з шуканою ознакою. Використаємо її.

In [17]:
```
def kind(extension):
    if extension in CODE_LIST:
        return "C"
    if extension in WHITE_LIST:
        return "W"
    return "O"
```

Додамо до таблиці стовпчик з ознаками розширень файлів.

Зараз розширення є індексом. За допомогою методу метод `reset_index` (документація тут) зробимо його стовпчиком таблиці.

Не забувайте виконувати перетворення "на місці".

In [18]:
```
task1.reset_index(inplace=True)
task1.head(3)
```

Out[18]:

	ext	cnt
0	.py	9499
1	.cs	692
2	.json	421

Тепер вже додаємо стовпчик з ознаками.

▶ Підказки

In [19]:
```
task1["kind"] = task1["ext"].apply(kind)
task1.head(3)
```

Out[19]:

	ext	cnt	kind
0	.py	9499	C
1	.cs	692	C
2	.json	421	W

За допомогою методу `set_index` (документація тут) знов зробимо розширення індексом таблиці.

Тут: Значенням параметра `keys` має стати ім'я стовпчика, що слід використати як індекс.

Не забувайте виконувати перетворення "на місці".

In [20]:
```
task1.set_index("ext", inplace=True)
task1.head(3)
```

Out[20]:

	cnt	kind
ext		
.py	9499	C
.cs	692	C
.json	421	W

Змінимо написи.

In [21]:
```
task1.index.name = "Розширення"
task1.columns = ["Кількість файлів", "Ознака"]
task1.head()
```

Out[21]:

	Кількість файлів	Ознака
Розширення		
.py	9499	C
.cs	692	C
.json	421	W
.txt	369	W
.js	158	C

In [22]:
```
task1.head(20)
```

Out[22]:

Розширення	Кількість файлів	Ознака
.py	9499	C
.cs	692	C
.json	421	W
.txt	369	W
.js	158	C
.h	146	C
.cpp	92	C
.c	45	C
.hpp	23	C
.ipynb	20	W
.html	13	W

Задача 2 За датами створення файлів для кожного місяця знайти таку інформацію: кількість файлів з розширеннями з "білого списку", їх сумарний розмір у кілобайтах, кількість їх рядків (враховувати тільки для файлів категорії C), середній розмір у байтах, середню кількість рядків.

Відповідну інформацію вивести на екран у табличному вигляді в порядку, оберненому до хронологічного.

Всі значення округлити та привести до цілого типу.

План:

1. Додати до початкової таблиці `data` стовпчик `kind`, що містить ознаки розширень.
2. Вибрати дані, що стосуються розширень з "білого списку".
3. Згрупувати вибрані дані в межах року та місяця.
4. За згрупованою інформацією обчислити потрібні агрегати.
5. Виконати перетворення стовпчиків та відсортувати.

Щоб за невдачі повторно не завантажувати дані, створимо їх копію.

In [23]:
```
data1 = data.copy()
```

1. Додати до початкової таблиці `data` стовпчик `kind`, що містить ознаки розширень.

Аналогічно тому, що виконувалось раніше, додати до таблиці `data1` стовпчик `kind`, що містить ознаки.

In [24]:
```
data1["kind"] = data1["ext"].apply(kind)
data1.head(3)
```

Out[24]:

	year	mon	ext	cnt	size	lines	kind
0	2025	2	.txt	19	60336	<NA>	W
1	2025	2	.h	5	2139	91	C
2	2025	2	.cpp	13	119213	3273	C

2. Вибрати дані, що стосуються розширень з "білого списку".

Запишіть вираз, що задає вибірку тільки інформації про розширення з "білого списку". Щоб далі не копіювати сам вираз, присвойте його значення змінній `selection`. (Це є зручно, коли експериментуємо, у продакшн потреби в цій змінній насправді не буде, замість неї можна буде використовувати вираз безпосередньо.)

▶ Підказка

In [25]:
```
selection = data1[data1["kind"] != "O"]
selection.head()
```

Out[25]:

	year	mon	ext	cnt	size	lines	kind
0	2025	2	.txt	19	60336	<NA>	W
1	2025	2	.h	5	2139	91	C
2	2025	2	.cpp	13	119213	3273	C
3	2025	2	.json	33	205855	<NA>	W
4	2025	2	.c	4	110480	3000	C

3. Згрупувати вибрані дані за ознаками в межах року та місяця.

   Допоможе метод `groupby` документація тут.

Групувати слід за роком та місяцем (параметр `by` має бути списком). Оскільки далі будемо обчислювати статистики для розміру та кількості рядків, то перед групуванням з таблиці слід вибрати стовпчики, за якими групуємо і за якими обчислюємо статистики.

In [26]:
```
task2 = selection[["year", "mon", "cnt", "size", "lines"]].groupby(by=["y
```

Далі необхідно знайти кількість файлів за кожен місяць, їх сумарний та середній розмір, а також сумарну та середню кількість рядків.

У результаті групування отримався об'єкт `DataFrameGroupBy` (документація тут).

До нього застосовний метод `agg`, що обчислює статистики (агрегатні характеристики) (документація тут).

Для стовпчика `cnt` будемо шукати суму значень (`sum`), для стовпчика `size` будемо шукати суму значень та середнє значення (`mean`), для стовпчика `lines` будемо шукати суму значень та середнє значення.

Першим параметром методу `agg` зробимо словник, що задає які статистики для яких стовпчиків слід обчислювати. Ключами словника мають бути імена стовпчиків (рядки), а значеннями списки статистик або окремі статистики (статистики також задаємо рядками-іменами)).

Результат агрегування зробимо значенням змінної `res2`.

In [27]:
```
res2 = task2.agg({"cnt": "sum", "size": ["sum", "mean"], "lines": ["sum",
res2.head()
```

Out[27]:

		cnt	size		lines	
		sum	sum	mean	sum	mean
year	mon					
2025	2	74	498023	9.960460e+04	6364	2121.333333
	3	2715	29469720	3.683715e+06	696121	116020.166667
	4	19	87734	1.462233e+04	1935	483.75
	5	91	824131	1.177330e+05	9436	2359.0
	6	62	3668059	9.170148e+05	33278	16639.0

Оскільки відбувалось групування по кільком стовпчикам та для окремих стовпчиків обчислювалось кілька характеристик, то отримали складені індекси (multindex) і рядків, і стовпчиків.

Поточний індекс стовпчиків.

In [28]:
```
res2.columns
```

Out[28]:
```
MultiIndex([(  'cnt',  'sum'),
            ( 'size',  'sum'),
            ( 'size', 'mean'),
            ('lines',  'sum'),
            ('lines', 'mean')],
           )
```

Перейдемо до простого індексу стовпчиків.

In [29]:
```
res2.columns = res2.columns.to_flat_index()
res2.columns
```

Out[29]:
```
Index([   ('cnt', 'sum'),   ('size', 'sum'),  ('size', 'mean'),
        ('lines', 'sum'), ('lines', 'mean')],
      dtype='object')
```

In [30]:
```
res2.head(3)
```

Out[30]:

year	mon	(cnt, sum)	(size, sum)	(size, mean)	(lines, sum)	(lines, mean)
2025	**2**	74	498023	9.960460e+04	6364	2121.333333
	3	2715	29469720	3.683715e+06	696121	116020.166667
	4	19	87734	1.462233e+04	1935	483.75

Змінимо назви стовпчиків.

In [31]:
```
res2.columns = [
    "Кількість",
    "Сумарний розмір",
    "Середній розмір",
    "Кількість рядків",
    "Середня кількість рядків",
]
res2.head(3)
```

Out[31]:

year	mon	Кількість	Сумарний розмір	Середній розмір	Кількість рядків	Середня кількість рядків
2025	**2**	74	498023	9.960460e+04	6364	2121.333333
	3	2715	29469720	3.683715e+06	696121	116020.166667
	4	19	87734	1.462233e+04	1935	483.75

Виразимо сумарний розмір у kb.

Для цього слід поділити значення на 1024, округлити. Додатково перетворимо на цілий тип (метод `astype` дозволяє виконувати перетворення типів, документація тут). Також не забудемо змінити підпис стовпчика.

Переходимо до кілобайтів.

In [32]:
```
res2["Сумарний розмір"] = (res2["Сумарний розмір"] / 1024).round().astype
```

Змінюємо підпис стовпчиків

In [33]:
```
res2.rename(
    columns={
        "Сумарний розмір": "Сумарний розмір, kb",
        "Середній розмір": "Середній розмір, b",
    },
    inplace=True,
)
```

In [34]:
```
res2.head(3)
```

Out[34]:

		Кількість	Сумарний розмір, kb	Середній розмір, b	Кількість рядків	Середня кількість рядків
year	**mon**					
2025	**2**	74	486	9.960460e+04	6364	2121.333333
	3	2715	28779	3.683715e+06	696121	116020.166667
	4	19	86	1.462233e+04	1935	483.75

Округлити середній розмір та середню кількість рядків, перейти до цілих значень.

In [35]:
```
res2.info()
```

```
<class 'pandas.DataFrame'>
MultiIndex: 12 entries, (np.int64(2025), np.int64(2)) to (np.int64(2026),
np.int64(2))
Data columns (total 5 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Кількість                 12 non-null     int64
 1   Сумарний розмір, kb       12 non-null     int64
 2   Середній розмір, b        12 non-null     float64
 3   Кількість рядків          12 non-null     Int64
 4   Середня кількість рядків  12 non-null     Float64
dtypes: Float64(1), Int64(1), float64(1), int64(2)
memory usage: 721.0 bytes
```

Ось тільки тепер замінимо відсутні значення кількості рядків нулями!

Якщо б це було зроблено раніше, то середня кількість рядків була б обрахована неправильно (на неї вплинули б нульові значення замість відсутніх).

▶ Підказка

In [36]:
```
res2.fillna({"Кількість рядків": 0}, inplace=True)
res2.info()
```

```
<class 'pandas.DataFrame'>
MultiIndex: 12 entries, (np.int64(2025), np.int64(2)) to (np.int64(2026),
np.int64(2))
Data columns (total 5 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Кількість                 12 non-null     int64
 1   Сумарний розмір, kb       12 non-null     int64
 2   Середній розмір, b        12 non-null     float64
 3   Кількість рядків          12 non-null     Int64
 4   Середня кількість рядків  12 non-null     Float64
dtypes: Float64(1), Int64(1), float64(1), int64(2)
memory usage: 721.0 bytes
```

Тепер можна перетворювати типи стовпчиків.

In [37]:
```
fname = 'Середній розмір, b'
res2[fname] = res2[fname].round().astype("int64")
```

```
fname = 'Середня кількість рядків'
res2[fname] = res2[fname].round().astype("int64")
```

In [38]:
```
res2
```

Out[38]:

year	mon	Кількість	Сумарний розмір, kb	Середній розмір, b	Кількість рядків	Середня кількість рядків
2025	**2**	74	486	99605	6364	2121
	3	2715	28779	3683715	696121	116020
	4	19	86	14622	1935	484
	5	91	805	117733	9436	2359
	6	62	3582	917015	33278	16639
	8	248	1705	582021	16209	16209
	9	178	1886	241412	35221	7044
	10	552	5968	1018515	121940	30485
	11	592	6253	800425	165363	33073
	12	6315	111624	11430328	2635169	376453
2026	**1**	60	1608	329338	1519	760
	2	572	4002	585419	48231	12058

Відсортуємо в порядку, оберненому до хронологічного.

У даному випадку слід сортувати індекс. Допоможе метод `sort_index` (документація тут).

In [39]:
```
res2.sort_index(ascending=[False, False], inplace=True)
res2
```

Out[39]:

year	mon	Кількість	Сумарний розмір, kb	Середній розмір, b	Кількість рядків	Середня кількість рядків
2026	**2**	572	4002	585419	48231	12058
	1	60	1608	329338	1519	760
2025	**12**	6315	111624	11430328	2635169	376453
	11	592	6253	800425	165363	33073
	10	552	5968	1018515	121940	30485
	9	178	1886	241412	35221	7044
	8	248	1705	582021	16209	16209
	6	62	3582	917015	33278	16639
	5	91	805	117733	9436	2359
	4	19	86	14622	1935	484
	3	2715	28779	3683715	696121	116020
	2	74	486	99605	6364	2121

Частину 2 зроблено!